

## SPOT CHECK

SKATESPOTS FIELD MANUAL · Nº02

# DATA — SHAPE & THE MODEL

Everything reasoned through while designing the `Spot` type and building `mockSpots`: what a data shape is, why it's a *contract*, what actually survives a trip through JSON, and how one skate spot lives at three different layers — the world, the wire, and the database.

11 chapters

3 parts

Self-test

Phase 1 — The Spot type

## ► CONTENTS (11 CHAPTERS + APPENDICES)

### | Serialization

What actually travels between server and browser

## 01 What "serialization" actually means

### ● CONCEPT

KEYWORDS

serialization

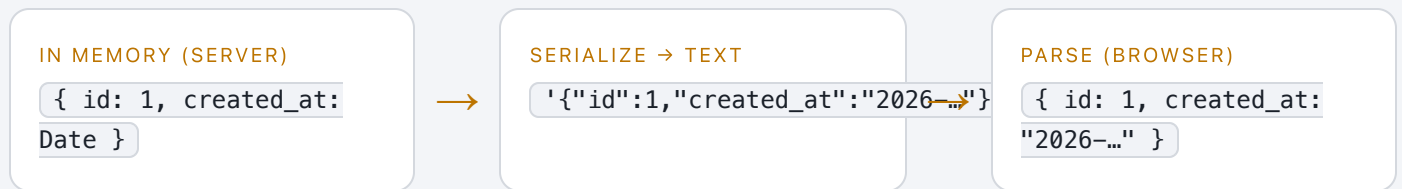
deserialization

wire shape

### — WHAT

A `Spot` lives in memory as a rich JavaScript object — it can hold a `Date`, nested objects, whatever. A network can't send an "object," only **bytes**, which for a web API means **text**.

## — HOW



In JavaScript this pair is `JSON.stringify()` (object  $\rightarrow$  text) and `JSON.parse()` (text  $\rightarrow$  object).

## — WHY

**Serialization** is turning an in-memory object into flat text; **deserialization** (parsing) rebuilds an object from it on the other end. Your API does the first; TanStack Query does the second for you in Phase 7.

**The one law of UI-first:** a mock `Spot` must look exactly like the object that comes out of `JSON.parse()` on the client — *not* the object that existed on the server before serialization. Those two are not the same, and Chapters 03–04 are where they differ.

### ◆ IDEA TO CONSIDER

This is why mock data is only useful if it's shaped like the *wire*, not like your ideal in-memory model. A mock that's more "correct" than what the real API will actually send is a mock that lies to you the day you connect the real backend.

## 02 JSON's tiny type vocabulary

### ● CONCEPT

### ● GOTCHA

#### KEYWORDS

JSON types

undefined vs null

## — WHAT

JSON only knows **six** value types. Everything in a `Spot` has to reduce to one of them, or it gets transformed / dropped on the way out.

## — HOW

| JSON TYPE            | YOUR SPOT FIELD                                                             | NOTE                                         |
|----------------------|-----------------------------------------------------------------------------|----------------------------------------------|
| <code>string</code>  | <code>name</code> , <code>city</code> , <code>created_at</code>             | Text in double quotes only                   |
| <code>number</code>  | <code>id</code> , <code>rating</code> , <code>lat</code> , <code>lng</code> | IEEE-754 double — Ch 05                      |
| <code>boolean</code> | <code>is_skateable</code>                                                   | <code>true</code> / <code>false</code>       |
| <code>null</code>    | an absent <code>photo</code>                                                | The only "empty" — no <code>undefined</code> |
| <code>array</code>   | <code>features</code>                                                       | Ordered list                                 |
| <code>object</code>  | <code>lat_lng</code>                                                        | Nested key/value                             |

- **No `Date`** — becomes a string (Ch 03).
- **No `undefined`** — a key whose value is `undefined` is *silently omitted* entirely. This is why "optional" and "null" are different things (Ch 08).
- **No `enums`, `classes`, `functions`, `Map` / `Set`** — flatten to plain objects or vanish.
- **No `BigInt`** — `JSON.stringify` throws on it.

## — WHY

JSON was designed as a minimal, language-agnostic text format — it deliberately doesn't try to represent every JS runtime type, only the shapes common to essentially every language's data structures.

**Remember:** "JSON is text with six shapes." Anything richer than those six is your responsibility to encode on the way out and rebuild on the way in.

### ◆ IDEA TO CONSIDER

Try `JSON.stringify({ a: undefined, b: null, c: new Date() })` in a console once. Watch `a` disappear, `b` stay, `c` turn into a string — that single line is this whole chapter, and it explains why "optional" (Ch 08) and "transformed" (Ch 03) are two completely different concerns.

## 03 Dates don't survive the trip

● BUG-IN-WAITING

● CONCEPT

KEYWORDS

Date.toJSON()

ISO-8601

re-hydration

### — WHAT

`created_at` is typed `string`, not `Date` — that looks odd at first (it is a date), but it's correct.

### — HOW

```
// before – in memory
{ id: 1, created_at: new Date() }

// after – JSON.stringify()
{ "id": 1, "created_at": "2026-08-02T14:30:00.000Z" }
```

Keep the transport type a `string`. Convert to a `Date` only at the moment you need date *math* or formatting, right where you use it:

```
const created = new Date(spot.created_at); // re-hydrate on demand
created.toLocaleDateString();             // "8/2/2026"
```

### — WHY

A `Date` object has a hidden `.toJSON()` method. When `JSON.stringify` hits it, it calls that method, which returns an **ISO-8601 string**. So by the time a spot reaches the browser, `created_at` is — and only ever was — a **string**. There is no `Date` on the client unless you rebuild one.

**The honesty rule for mock data:** because the real API sends a full ISO timestamp, mock `created_at` should be a full ISO string too — not a date-only string. A mock shaped differently from the API is a mock that lies, and the lie surfaces the day you swap in the real backend.

#### ◆ IDEA TO CONSIDER

This is the exact same lesson as Study Sheet 05, Item 5 (naming mismatches across a language/file boundary) wearing a different costume: a JS-native type (`Date`) simply doesn't exist on the other side of a serialization boundary. Any type richer than JSON's six needs an explicit, deliberate re-hydration step — it never survives by accident.

## 04 Location on the wire — the lat/lng order trap

### ● CONCEPT

### ● GOTCHA

#### KEYWORDS

named object vs. array

coordinate order

#### — WHAT

`lat_lng: { lat: number; lng: number }` was chosen over two loose columns or a raw array — a small choice guarding one of the most bug-prone corners of the project.

#### — HOW

| SHAPE                 | LOOKS LIKE                             | RISK                          |
|-----------------------|----------------------------------------|-------------------------------|
| Two fields            | <code>lat: 42.3, lng: -71.0</code>     | Location can get half-updated |
| Array                 | <code>[-71.0, 42.3]</code>             | <b>Which one is which?</b>    |
| <b>Named object ✓</b> | <code>{ lat: 42.3, lng: -71.0 }</code> | Self-documenting — chosen     |

| TOOL                                   | ORDER IT EXPECTS                          |
|----------------------------------------|-------------------------------------------|
| <b>Leaflet</b> (Phase 3)               | <code>[lat, lng]</code> — latitude first  |
| <b>GeoJSON / PostGIS</b> (Phase 6, 10) | <code>[lng, lat]</code> — longitude first |

#### — WHY

The database stores location as a PostGIS *point*, which serializes to plain numbers — the only question is order, and different tools disagree about it *silently*. Both values are just numbers, so nothing errors; your pin just lands in the wrong ocean.

**Boston is** `lat 42.36, lng -71.06` . Swap them and you get a point off the coast of Western Sahara — a bug that parses fine and points nowhere real. Naming the fields `lat` and `lng` means the order can never be ambiguous at the call site.

#### ◆ IDEA TO CONSIDER

Anywhere two independently-built tools have to agree on the meaning of a bare positional value (array order, argument order, column order), that agreement is exactly the kind of thing that silently drifts. A named field is cheap insurance against a whole category of "everything type-checks, nothing works" bugs.

## II Modeling the Spot

The specific decisions baked into the type

# 05 id: number or string?

### ● CONCEPT

KEYWORDS `primary key` `enumerability`

#### — WHAT

`id: number` was chosen. A real fork in the road — it ripples into React `key` props, the `/api/spots/[id]` route, and the Prisma primary key.

#### — HOW

|             | <code>NUMBER</code> (AUTO-INCREMENT)               | <code>STRING</code> (CUID / UUID)         |
|-------------|----------------------------------------------------|-------------------------------------------|
| DB default  | <code>serial</code> / <code>autoincrement()</code> | <code>cuid()</code> / <code>uuid()</code> |
| Readability | Great — <code>1, 2, 3</code>                       | Opaque — <code>clx1a...</code>            |
| Guessable?  | Yes — <code>/spots/2</code> invites enumeration    | No — practically unguessable              |
| Made where? | By the database on insert                          | Can be made on the client too             |

## — WHY

For a learning project, `number` is a perfectly good call — readable Prisma Studio rows, friendly URLs. The trade-off accepted: sequential ids leak "how many spots exist" and are trivially enumerable. If this ever went public with private data, that's when to revisit it.

**Contract consequence:** whatever you pick, mock ids must be that type. `id: 1` and `id: "1"` are different contracts, and `===` comparisons in click handlers will care.

### ◆ IDEA TO CONSIDER

The real axis isn't "which is more readable" — it's *"who assigns this value, and when?"* A database-assigned sequential id and a client-generatable uuid solve genuinely different problems (offline-first apps often need the latter, so an id can exist before the server has ever seen the record).

## 06 Union types as enums

### ● CONCEPT

KEYWORDS `string-literal union` `runtime validation`

## — WHAT

Instead of typing `spot_type: string`, the exact allowed values are named directly.

## — HOW

```
export type SpotType = "park" | "street" | "diy";
export type Status   = "active" | "pending" | "closed";
export type Feature   = "ledge" | "rail" | "plaza" | "stairs" | "skatepark";
```

## — WHY

A **string-literal union** turns a whole category of runtime bugs into compile-time errors. `spot_type: "street"` (typo) is a red squiggle immediately, not a filter that silently matches nothing later. The editor also autocompletes the legal values.

**This is a contract, not a convenience.** A later filter comparing `spot.spot_type === activeFilter` is comparing against these exact strings, and the Postgres/Prisma enum must list the same values, character for character.

**The watch-out:** a union only guards code the compiler can see. Data arriving from the network is typed `any` until validated — a DB row with `spot_type: "vert"` would sail past TypeScript. That's exactly why runtime validation (e.g. Zod) exists: to re-check unions at the boundary the compiler can't reach.

#### ◆ IDEA TO CONSIDER

This is the compile-time half of the same exhaustiveness idea from Study Sheet 05, Item 2 ( `Record<K,V>` ) — a closed set of legal values, enforced by the type system. The gap it leaves (runtime data isn't automatically checked) is precisely the seam where a schema-validation library earns its keep.

## 07 `spot_type` vs. features — category vs. attributes

### ● PATTERN

KEYWORDS

`category vs. attribute`

`data modeling`

### — WHAT

The roadmap floated one flat list — `park | street | ledge | rail | stairs | bowl | DIY`. It was split into **two** dimensions instead.

### — HOW

SPOT\_TYPE — WHAT KIND OF PLACE

FEATURES — WHAT'S THERE



"park" | "street" | "diy"

exactly one per spot

+

("ledge" | "rail" | "plaza" | ...)[ ]

many per spot

### — WHY

Those original values weren't the same *kind* of thing. "Street" is a category; "ledge" and "rail" are things a street spot *has* — and can have several at once. Cramming them into one field forces a false choice. This is the classic modeling question: **"is this a single category, or a set of attributes?"** Single category → one enum field. Set of independent traits → an array (or later, a related table).

**The seam for later:** this two-axis design maps naturally to the database — `spot_type` becomes an enum column, `features` becomes an array column or a related table. The flexibility is built in for free.

### ◆ IDEA TO CONSIDER

This "one category vs. many attributes" question recurs everywhere in modeling — user roles vs. permissions, order status vs. order tags, a blog post's type vs. its labels. If you ever feel the urge to put the same value in both fields, that's the signal the two axes have blurred back together.

## 08 Optional fields = the nullability contract

### ● CONCEPT

### ● GOTCHA

KEYWORDS

optional (?)

nullable column

### — WHAT

`description?` and `photo?` are optional; the rest are required. Each `?` is a promise to the rest of the app: *"this field might not be here — you must handle its absence."*

### — HOW

| ON THE WIRE                | IN TS AFTER PARSE                | MEANING                                  |
|----------------------------|----------------------------------|------------------------------------------|
| key absent                 | <code>photo === undefined</code> | "never set" — what <code>?</code> models |
| <code>"photo": null</code> | <code>photo === null</code>      | "explicitly empty" — DB NULL             |
| <code>"photo": ""</code>   | <code>photo === ""</code>        | "set, to nothing" — a real string!       |

**The mock uses** `photo: ""`. That's a *present, empty* string — a different case from the optional-and-absent that `photo?` describes. Pick one convention and make the mock and the API agree.

#### — WHY

Every optional field is a branch a component must write: `{spot.photo ? <img .../> : <Placeholder/>}`. Deciding nullability now, at the contract, is deciding how many of those branches exist. It also lines up with the database: an optional field is a **nullable column**; a required one is `NOT NULL`.

#### ◆ IDEA TO CONSIDER

Three states masquerading as "empty" (absent, `null`, `""`) is a very common source of subtle bugs in any typed system talking to an untyped wire format. Deciding — in writing, on purpose — which one your codebase means by "no value" heads off a whole class of "why doesn't my falsy check work" debugging later.

## 09 Naming consistency & type/data lockstep

● BUG AVOIDED

● CONCEPT

KEYWORDS `snake_case` vs `camelCase` `excess property check`

#### — WHAT

The shape briefly mixed conventions — `spot_type`, `lat_lng`, `is_skateable` in `snake_case` but `createdAt` in `camelCase`. It was unified on `snake_case` — and,

crucially, changed in **both** the type *and* the data at once.

#### — HOW

Deliberately rename one field in mock data but not the type, save, and read the two errors TypeScript gives (missing key + excess key). Fix the type. That error pair is the lockstep lesson.

#### — WHY

The type file and the mock file are two halves of **one contract**. Rename a field in the data but not the type, and TypeScript flags both: the object is missing a required old key and carries an unknown new one. That's the discipline — *a shape change is never done until the type and every producer of that shape agree*.

**Why check the compiler at all:** a clean `tsc` is the cheap proof that type and data still describe the same thing. Treat a green compile as a passing test of your data shape.

JavaScript's own convention is `camelCase`; `snake_case` keys are common in APIs and match SQL column names. Either is fine — **consistency is the whole point**. If Postgres columns end up `snake_case` and some JS wants `camelCase`, the translation happens in exactly one place: the serialization boundary (Ch 11).

#### ◆ IDEA TO CONSIDER

"Type and data move in lockstep" is a narrower instance of single source of truth (Study Sheet 05, Item 4) — the type file and the data file are two independently-editable places describing the same contract, and only the compiler is actually watching that they still agree.

## III Entity → Model

One spot, living at three layers of the stack

# 10 One spot, three representations

## ● PATTERN

KEYWORDS

DTO

domain entity

data model

### — WHAT

"A skate spot" is not one thing in the codebase — it's the *same concept* wearing three different costumes depending on where it stands.

### — HOW

| LAYER               | NAME FOR IT                      | WHAT IT IS                                        | LOCATION = ?                  |
|---------------------|----------------------------------|---------------------------------------------------|-------------------------------|
| 1.<br><b>Entity</b> | Domain entity                    | The real ledge by the water. A concept, not code. | a physical place              |
| 2. <b>Wire</b>      | DTO / the <code>Spot</code> type | The JSON the API sends & the UI renders.          | <code>{ lat, lng }</code>     |
| 3. <b>Store</b>     | Data model / schema              | The Prisma model & Postgres table it lives in.    | <code>geography(Point)</code> |

```
// 2 · DTO (types/spot.ts) – the contract, Chapters 05–09
{ id: 3, name: "Copley Library", spot_type: "street",
  features: ["plaza"], lat_lng: { lat: 42.35, lng: -71.08 },
  created_at: "2026-08-02T...Z" }

// 3 · DATA MODEL (schema.prisma → Postgres)
model Spot {
  id          Int          @id @default(autoincrement())
  name        String
  spot_type   SpotType     // Postgres enum
  location    Unsupported("geography(Point,4326)")
  created_at  DateTime @default(now())           // timestamptz
}
```

### — WHY

Confusing the layers is the root of most data bugs; separating them is most of software architecture.

**Why UI-first works because of this:** layer 2 (the DTO) is built first, by hand, against mock data. Layer 3 (the model) comes later and is shaped to *match* what layer 2 already promised — get the middle layer right and the bottom layer just conforms to it.

◆ **IDEA TO CONSIDER**

This three-layer split (entity / DTO / persistence model) is one of the most transferable ideas in this whole manual — it's the same shape as MVC's model layer vs. its view-model, or a domain-driven-design "aggregate" vs. its database row. Once you can name which layer a piece of code is operating at, a lot of "where does this logic belong" confusion resolves itself.

# 11 The mapping — and where the transformation lives

● **CONCEPT** ● **GOTCHA**

KEYWORDS impedance mismatch route handler

## — WHAT

Every field either passes through unchanged or gets **transformed** as it crosses from the database (layer 3) to the wire (layer 2).

## — HOW

| DTO FIELD (CH 05–09)      | POSTGRES / PRISMA                                              | TRANSFORM?   |
|---------------------------|----------------------------------------------------------------|--------------|
| <span>id: number</span>   | <span>Int @id</span><br><span>@default(autoincrement())</span> | pass-through |
| <span>name: string</span> | <span>String</span> / <span>text</span>                        | pass-through |

| DTO FIELD (CH 05–09)              | POSTGRES / PRISMA                               | TRANSFORM?                            |
|-----------------------------------|-------------------------------------------------|---------------------------------------|
| <code>spot_type: SpotType</code>  | Postgres <code>enum</code>                      | same values, different world          |
| <code>features: Feature[]</code>  | <code>text[]</code> or related table            | array ↔ column/rows                   |
| <code>lat_lng: {lat, lng}</code>  | <code>geography(Point, 4326)</code>             | <b>real transform</b> — Ch 04         |
| <code>created_at: string</code>   | <code>DateTime</code> / <code>timestampz</code> | <b>ISO string ↔ timestamp</b> — Ch 03 |
| <code>description?: string</code> | nullable <code>text</code>                      | optional ↔ NULL — Ch 08               |



### — WHY

The transform happens neither in the browser nor the database — in the **server route handler**, the one place both worlds are visible at once. This gap between "how it's stored" and "how it's used" has a name: **impedance mismatch**. The server boundary's job is to absorb it, so the UI never sees a `geography` blob and the database never sees a JS `Date`.

**The PostGIS reality check:** Prisma can't fully model a `geography` column — it gets declared `Unsupported(...)` and read with a raw, parameterized query ( `ST_X` / `ST_Y` ). That's not Prisma failing; it's the exact seam where layer 3 refuses to auto-map to layer 2, and a person has to step in.

### ◆ IDEA TO CONSIDER

For each row in the mapping table, the honest answer is always either "pass-through" or "transform" — and the three transforms here (location, date, optional/NULL) are precisely the three things earlier chapters drilled. That's not a coincidence: knowing what has to transform at a boundary is the whole reason to understand the layers in the first place.

# A Glossary

## Serialization / deserialization

Turning an in-memory object into flat text for transport ( `JSON.stringify` ), and rebuilding an object from that text ( `JSON.parse` ).

## Wire shape

The exact object shape that comes out of `JSON.parse()` on the receiving end — the only shape a client ever actually sees, regardless of what existed in memory before serialization.

## String-literal union

A TypeScript type naming the exact legal string values ( `"park" | "street" | "diy"` ) instead of the general `string` — compile-time enum-like safety.

## Nullable column

A database column allowed to hold `NULL` — the storage-layer counterpart of a TypeScript optional ( `?` ) field.

## DTO (Data Transfer Object)

The shape of data as it crosses a boundary (like an API response) — distinct from both the real-world entity it represents and the database model that stores it.

## Impedance mismatch

The general gap between how data is stored (e.g. a relational/geographic database) and how it's used (e.g. a JS object) — something has to translate between the two.

---

# B Self-test

Say the answer aloud *before* opening each — the explaining is what makes it stick.

Q1 Why is `created_at` typed `string` and not `Date` ? ▶

Q2 What happens to a key whose value is `undefined` when you `JSON.stringify` ? ▶

Q3 Why is a named `{lat, lng}` object safer than `[42.36, -71.06]` ? ▶

Q4 Why split `spot_type` from `features` instead of one flat enum? ▶

Q5 Name the three ways "no photo" can appear, and which one `photo?` describes. ▶

Q6 You rename a field in the mock but not the type. What does TypeScript say? ▶

Q7 Name the three layers a single spot lives at. ▶

Q8 Which fields *transform* between database and wire, and where does that genuinely transform happen? ▶

---

## C Study links — go deeper

### — SERIALIZATION & JSON

↗ [MDN — JSON.stringify \(what it transforms & drops\)](#)

↗ [MDN — Date.toJSON](#)

### — TYPING THE SHAPE

↗ [TypeScript — Everyday Types](#)

↗ [Zod — runtime validation at the network boundary](#)

### — THE DATA MODEL & GEO

↗ [Prisma — data model & ids](#)

↗ [GeoJSON RFC 7946 \(coordinate order\)](#)

### — ARCHITECTURE CONCEPTS

↗ [Fowler — Data Transfer Object](#)



## **SPOT CHECK**

SkateSpots Field Manual Nº02 · built while designing the Spot type and mockSpots.